DESARROLLO DE APLICACIONES WEB · SEMANA 12

Práctica: Manejadores y plantillas en Django

Resolución de las consultas ORM y evidencia de ejecución en Django Shell.

Entorno: Python 3.12 · Django 6.1.1 · SQLite · Proyecto comercial
Aplicación: `catalogo`

Datos de la práctica

Se crearon las categorías solicitadas y cinco productos de ejemplo para probar filtros, ordenamiento, búsquedas, relación inversa y existencia de productos sin stock.

PK	Producto	Precio	Stock	Categoría
1	Memoria USB 64 GB	$12.50	15	Accesorios
2	Memoria USB 128 GB	$24.99	0	Almacenamiento
3	Hub USB 4 puertos	$59.90	8	Accesorios
4	SSD externo 1 TB	$89.99	4	Almacenamiento
5	Mouse inalámbrico	$35.00	20	Accesorios

La migración del proyecto también aporta las categorías de ejemplo “tv” y “Laptop”. El precio de venta del modelo se llama `precio`.

Creación desde Django Shell

```
from decimal import Decimal
from django.utils import timezone
from catalogo.models import Categoria, Producto

ahora = timezone.now()
accesorios = Categoria.objects.create(
    nombre="Accesorios", descripcion="Accesorios para equipos",
    created_at=ahora, updated_at=ahora
)
almacenamiento = Categoria.objects.create(
    nombre="Almacenamiento",
    descripcion="Memorias y unidades de almacenamiento",
    created_at=ahora, updated_at=ahora
)
# Se crearon con Producto.objects.create(...) los cinco productos de la tabla.
```

Consultas solicitadas y resultados

#	Operación	Comando ORM	Resultado
1–2	Crear categorías	`Categoria.objects.create(...)`	Accesorios (pk 3) y Almacenamiento (pk 4)
3	Agregar productos	`Producto.objects.create(...)`	5 productos creados
4	Consultar todos	`Producto.objects.all()`	5 registros
5	Obtener pk=5	`Producto.objects.get(pk=5)`	Mouse inalámbrico
6	Stock > 10	`Producto.objects.filter(stock__gt=10)`	Memoria USB 64 GB; Mouse inalámbrico
7	Precio ≥ $50	`Producto.objects.filter(precio__gte=50)`	Hub USB 4 puertos; SSD externo 1 TB
8	Nombre contiene USB	`Producto.objects.filter(nombre__icontains="USB")`	Hub USB 4 puertos; Memoria USB 128 GB; Memoria USB 64 GB
9	Precio descendente	`Producto.objects.order_by("-precio")`	SSD; Hub; Mouse; USB 128 GB; USB 64 GB
10	Productos de Accesorios	`Producto.objects.filter(categoria__nombre="Accesorios")`	Hub USB 4 puertos; Memoria USB 64 GB; Mouse inalámbrico
11	Relación inversa	`categoria.productos.all()`	3 productos de Accesorios
12	Contar productos	`Producto.objects.count()`	5
13	Stock igual a cero	`Producto.objects.filter(stock=0).exists()`	True

Transcripción de resultados

```
1-2. CATEGORÍAS CREADAS: Accesorios (id 3), Almacenamiento (id 4)
3. PRODUCTOS CREADOS: 5
4. TODOS LOS PRODUCTOS: 5 registros
5. PRODUCTO pk=5: Mouse inalámbrico
6. STOCK MAYOR QUE 10: ['Memoria USB 64 GB', 'Mouse inalámbrico']
7. PRECIO MAYOR O IGUAL A 50: ['Hub USB 4 puertos', 'SSD externo 1 TB']
8. NOMBRES QUE CONTIENEN USB: ['Hub USB 4 puertos', 'Memoria USB 128 GB', 'Memoria USB 64 GB']
9. ORDEN DESCENDENTE: SSD externo 1 TB ($89.99), Hub USB 4 puertos ($59.90),
   Mouse inalámbrico ($35.00), Memoria USB 128 GB ($24.99), Memoria USB 64 GB ($12.50)
10. PRODUCTOS DE ACCESORIOS: 3
```

```
11. categoria.productos.all(): 3 productos
12. TOTAL DE PRODUCTOS: 5
13. ¿EXISTEN PRODUCTOS CON STOCK 0?: True
```

Todas las consultas y escrituras anteriores se ejecutaron con el administrador ORM `objects` desde `python manage.py shell`.

Implementación de plantillas y CRUD

La vista de categorías consulta los registros de la base de datos. Se configuró una plantilla base heredable, un menú incluido y los bloques para el contenido de cada página.

- `templates/base.html` y `templates/menu.html`.
- `catalogo/templates/categoria/lista.html`, `add.html` y `update.html`.
- Formulario `CategoriaForm` con validación y Django Crispy Forms / Tailwind.
- Crear, editar y eliminar categorías, con token CSRF y eliminación protegida por POST.
- Mensajes de éxito en español y respuesta 404 al editar registros inexistentes.

Rutas disponibles

Ruta	Acción
`/categoria/`	Listado
`/categoria/add/`	Crear categoría
`/categoria/<id>/update/`	Actualizar categoría
`/categoria/<id>/delete/`	Eliminar categoría (POST)

Verificación

Django system check: sin problemas.

Migraciones: no hay cambios de modelo pendientes.

Pruebas: 11 correctas; incluye alta, actualización, eliminación, validación de formularios, listado vacío, modelos relacionados y borrado en cascada.

Fin de la evidencia · Proyecto comercial, Semana 12